

IA PARA DESENVOLVEDORES [T1]

Situação de Aprendizagem (Projeto Avaliativo) - Módulo 1 - Semana 08

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	2
4. REQUISITOS DA APLICAÇÃO	4
5. ROTEIRO DA APLICAÇÃO	5
5.1. FORMATO DO SISTEMA	5
5.2. DOCUMENTAÇÃO NO README.MD	6
5.3. USO DO QUADRO DO GITHUB	7
5.4. USO DO REPOSITÓRIO NO GITHUB	8
5.5. GRAVAÇÃO DE VÍDEO	9
6. CRITÉRIOS DE AVALIAÇÃO	10
7. CHECKLIST FINAL DE ENTREGA	13

1. CONTEXTUALIZAÇÃO

Times de desenvolvimento lidam diariamente com pressão para entregar software mais rápido, com mais qualidade e menor risco. Arquiteturas precisam ser documentadas, código precisa ser testado, pipelines precisam ser mantidos e repositórios precisam estar organizados. Quando isso acontece sem disciplina, acumulam-se dívidas técnicas difíceis de reverter: código sem testes, documentação desatualizada, pipelines frágeis e arquiteturas mal definidas.

Ferramentas de IA generativa transformaram profundamente esse cenário. Hoje é possível usar IA para especificar requisitos, gerar código, criar arquiteturas, produzir mockups, escrever testes unitários e E2E, gerar documentação técnica automática, criar pipelines de CI/CD e revisar decisões técnicas — tudo isso com prompts bem estruturados, engenharia de contexto adequada e padrões de prompting aplicados de forma intencional.

Neste projeto avaliativo, o grupo deverá desenvolver uma aplicação end-to-end com uso explícito de IA em todo o fluxo de desenvolvimento: do planejamento inicial à documentação final, passando pela geração de código, arquitetura, testes e pipeline. A solução deverá demonstrar que o grupo não apenas usou IA como ferramenta de autocomplete, mas que a integrou de forma estruturada, documentada e crítica em cada etapa do ciclo de vida do software.

Além da construção técnica da solução, o projeto também deverá evidenciar a capacidade do grupo de organizar o trabalho de forma colaborativa, distribuindo responsabilidades, registrando tarefas e acompanhando o progresso por meio do quadro do GitHub. Essa

organização deverá permitir rastrear a relação entre planejamento, execução, versionamento, prompts utilizados, refinamentos aplicados e artefatos entregues.

O objetivo não é criar um sistema de produção completo, mas sim uma aplicação funcional e bem documentada que demonstre domínio dos conteúdos estudados ao longo das 8 semanas: fundamentos de IA, engenharia de contexto, padrões de prompting, geração e refatoração de código, arquitetura assistida por IA, testes automatizados com IA, documentação automática e pipelines de CI/CD.

A solução deverá evidenciar que o grupo compreendeu como integrar IA ao fluxo de trabalho de um desenvolvedor de forma responsável, rastreável e tecnicamente justificada — não apenas gerando código, mas documentando decisões, salvando prompts, registrando refinamentos e entregando uma aplicação que outra pessoa consiga instalar, executar e entender.

2. DESAFIO

O grupo deverá desenvolver uma aplicação funcional, demonstrável e documentada, evidenciando não apenas o resultado final, mas também o processo de desenvolvimento apoiado por IA, incluindo prompts utilizados, decisões técnicas, ciclos de refinamento, testes, documentação e organização do trabalho no GitHub. Diante disso, o grupo deverá:

- Trabalhar com **Git**, entendendo que suas alterações podem impactar no desenvolvimento do projeto em geral;
- Atuar com processo de desenvolvimento de Software de forma colaborativa, utilizando o quadro do GitHub como ferramenta de organização, acompanhamento e gestão de atividades;
- Aplicar IA generativa ao longo do ciclo de desenvolvimento, desde a definição do problema até a geração de código, testes, documentação e pipeline;
- Utilizar engenharia de contexto e padrões de prompting para gerar, revisar e refinar artefatos técnicos do projeto;
- Planejar, implementar e documentar uma aplicação funcional, com arquitetura definida, funcionalidades principais, testes automatizados e pipeline de CI/CD;
- Analisar criticamente as saídas produzidas pela IA, registrando correções, refinamentos e decisões técnicas tomadas durante o desenvolvimento.

3. RESULTADOS ESPERADOS (ENTREGA)

A atividade será realizada em grupo. Cada grupo deverá ser formado por **3 integrantes**, e a formação dos trios deverá ser organizada na board do Miro disponibilizada para a turma:

https://miro.com/app/board/uXjVG1HILxU=

Cada grupo deverá definir um integrante para atuar como **líder técnico-organizacional**. Recomenda-se que essa pessoa seja alguém que se sinta confortável em apoiar a gestão das tarefas, orientar o grupo ao longo do desenvolvimento, acompanhar a evolução das entregas e contribuir tecnicamente durante a implementação.

O papel do líder não é centralizar o trabalho, mas facilitar a organização do grupo, apoiar a divisão das atividades, acompanhar o andamento das tarefas no **quadro do GitHub** e ajudar os demais integrantes quando surgirem dúvidas técnicas ou dificuldades no processo.

Mesmo assumindo esse papel de liderança, o líder também deverá participar ativamente do desenvolvimento, contribuindo com código, documentação, testes, prompts, revisão ou pipeline. Da mesma forma, os demais integrantes deverão assumir responsabilidades reais no projeto, executar suas atividades e registrar suas contribuições nos artefatos do grupo.

O objetivo é criar um ambiente colaborativo, no qual a liderança ajude a manter o grupo organizado e produtivo, sem substituir a participação individual de cada estudante.

O código deverá ser inserido e versionado no GitHub em modo privado e o planejamento e organização deverá ser realizado no quadro do GitHub. Os dois links deverão ser disponibilizados na tarefa "Projeto Avaliativo – M1.2", presente na semana 08 do AVA, até o dia 01/06/2026 às 15h.

O repositório deverá ser privado, com as seguintes pessoas adicionadas como colaboradoras:

- **NOME DO PROFESSOR RESPONSÁVEL** — nomeDoUsuárioDoProf.
- Operação LAB365 — lab365-operacao

Importante:

- Será considerado como data final de entrega a última atualização no repositório do GitHub. Não modifique o código após a entrega até receber sua nota.
- Não esqueça de submeter os links no AVA. Os links não submetidos terão penalidade na nota.
- Não serão aceitos projetos submetidos após a data limite ou alterados depois de entregues.

A entrega será realizada por grupo, considerando o repositório GitHub, o quadro do GitHub, a documentação, os testes, o pipeline e o vídeo de demonstração como artefatos principais do projeto.

Mesmo sendo uma entrega coletiva, cada estudante deverá participar ativamente do desenvolvimento e manter evidências de sua contribuição por meio de atividades no quadro do GitHub, commits, branches, pull requests, documentação, testes, prompts ou participação na demonstração.

O vídeo deverá ser publicado no YouTube como **não listado**, e o link deverá ser inserido no README.md do repositório.

Peso deste projeto: Avaliação M1.2 – 60% da nota do módulo.

4. REQUISITOS DA APLICAÇÃO

O grupo deverá desenvolver uma aplicação funcional em qualquer domínio de negócio, desde que o uso de IA esteja presente e documentado em todas as etapas do fluxo de desenvolvimento. O domínio fica a critério do grupo — o que será avaliado é a qualidade da integração de IA ao processo, não o tema escolhido.

Exemplos possíveis de domínio de aplicação:

- sistema de gerenciamento de tarefas ou projetos;
- API de cadastro e consulta com regras de negócio;
- aplicação de análise de dados com relatórios automáticos;
- ferramenta de automação de fluxos repetitivos;
- módulo de autenticação e autorização com validações;
- qualquer outro domínio funcional justificado e documentado pelo grupo.

O projeto deve atender aos seguintes requisitos funcionais:

- receber entradas do usuário em pelo menos uma forma definida pelo grupo (formulário, terminal, arquivo ou requisição de API);
- executar pelo menos duas funcionalidades principais com lógica de negócio real;
- ter sua arquitetura planejada com suporte de IA, com decisões documentadas;
- ter o código principal gerado e refinado com suporte de IA, com evidências de prompts e ciclos de refinamento;
- ter pelo menos uma refatoração documentada com suporte de IA (Clean Code, SOLID, otimização ou conversão arquitetural);
- possuir suíte de testes gerada com suporte de IA (unitários, de API ou E2E) com cobertura de cenários relevantes;
- possuir documentação técnica automática gerada com suporte de IA (docstrings, Swagger/OpenAPI, README ou diagrama);
- possuir pipeline de CI/CD configurado com suporte de IA (GitHub Actions ou equivalente), executando ao menos lint e testes automaticamente;
- gerar saída estruturada e validável compatível com o domínio escolhido;

- manter evidências dos prompts utilizados em cada etapa, salvos e organizados em pasta de documentação;
- demonstrar análise crítica sobre ao menos uma saída da IA que precisou ser corrigida ou refinada, com descrição do problema e da solução aplicada;
- demonstrar ao menos dois cenários de uso da aplicação.

O projeto deve atender aos seguintes requisitos técnicos:

- estar versionado em um repositório privado no GitHub, com os colaboradores obrigatórios adicionados;
- possuir README.md completo com instruções de instalação, configuração e execução;
- possuir .env.example sem chaves reais, caso utilize variáveis de ambiente;
- conter entradas e dados de exemplo para demonstração;
- conter exemplos de saída gerados pela aplicação;
- possuir documentação da arquitetura e das decisões técnicas tomadas com suporte de IA;
- possuir pasta docs/prompts/ com os prompts utilizados em cada etapa, organizados e identificados;
- possuir pipeline de CI/CD configurado e funcional no repositório;
- usar branches seguindo o fluxo definido neste documento;
- registrar e acompanhar as tarefas no quadro do GitHub.

As decisões de arquitetura e tecnologia ficam a critério do grupo. A avaliação considerará se essas decisões foram bem justificadas, se os componentes têm responsabilidades claras e se a solução demonstra o uso de IA como parceiro de desenvolvimento em todas as etapas — não apenas como gerador de código isolado.

5. ROTEIRO DA APLICAÇÃO

Abaixo seguem os requisitos a serem desenvolvidos.

5.1. FORMATO DO SISTEMA

O grupo poderá escolher o formato da aplicação, desde que a solução seja executável, demonstrável e documentada. São formatos aceitáveis:

- aplicação de linha de comando;

- API local com FastAPI, Flask ou similar;
- interface simples com Streamlit, Gradio ou similar;
- aplicação web simples (front-end + back-end);
- notebook executável (.ipynb), desde que acompanhado de instruções claras e pipeline configurado;
- composição local com scripts e serviços integrados.

O sistema deve aceitar ao menos uma forma de entrada definida pelo grupo:

- texto livre ou parâmetros via terminal;
- arquivo CSV, JSON, texto ou código como entrada;
- formulário em interface simples;
- requisição para API local.

A saída do sistema deve ser estruturada e alinhada ao domínio escolhido, permitindo identificar:

- quais entradas foram recebidas e processadas;
- quais funcionalidades foram executadas;
- quais resultados foram produzidos;
- quais partes foram geradas, refatoradas ou testadas com suporte de IA;
- quais limitações ou pontos de melhoria foram identificados.

Além da aplicação funcional, o projeto deve produzir evidências do processo de desenvolvimento com IA, incluindo: prompts salvos e organizados por etapa, comparativos antes/depois de refatorações, relatório de testes gerado automaticamente e documentação técnica produzida com apoio da IA.

Não há estrutura de pastas obrigatória. Porém, o repositório deve deixar claros os principais papéis arquiteturais da solução, incluindo: código-fonte principal, testes, documentação, prompts salvos, pipeline de CI/CD e exemplos de uso.

5.2. DOCUMENTAÇÃO NO README.MD

Crie um arquivo README.md no repositório do projeto no GitHub para documentar a solução, demonstrar as etapas em que a IA foi utilizada, apresentar as tecnologias escolhidas e instruir qualquer pessoa a executar o sistema.

O README.md deve conter obrigatoriamente:

- nome da aplicação e descrição do problema resolvido;

- quais ferramentas de IA foram utilizadas e em quais etapas (especificação, geração, arquitetura, refatoração, testes, documentação, pipeline);
- quais padrões de prompting foram aplicados, com exemplos ou referências aos arquivos salvos em docs/prompts/;
- diagrama ou descrição clara da arquitetura da solução;
- instruções completas de instalação, configuração e execução;
- descrição dos cenários de uso com exemplos de entrada e saída esperada;
- ao menos um caso documentado de saída da IA que precisou ser corrigida ou refinada, com descrição do problema e da solução aplicada;
- o que poderia ser melhorado ou expandido na solução;
- link do vídeo de demonstração no YouTube (não listado).

Algumas dicas para enriquecer o README como peça de portfólio:

- dê um nome criativo ao projeto e uma descrição clara do valor que ele entrega;
- insira um diagrama de arquitetura gerado com apoio de IA (draw.io, Mermaid, Figma ou similar);
- inclua prints ou gifs do sistema em funcionamento;
- destaque os padrões de prompting aplicados com um exemplo real de prompt usado;
- descreva o que você aprendeu ao usar IA em cada etapa do desenvolvimento.

5.3. USO DO QUADRO DO GITHUB

Crie um **quadro (board) no GitHub** para organizar o desenvolvimento do projeto utilizando a metodologia Kanban com as colunas: **Backlog, A Fazer, Em Andamento, Bloqueado, Em Revisão e Concluído**. A partir dos requisitos deste documento, organize as tarefas em cartões e os mova a cada etapa do desenvolvimento. Cada cartão deve ter descrição clara da tarefa, objetivo da implementação, critérios de conclusão e, quando aplicável, relação com a branch ou funcionalidade desenvolvida.

Faça um paralelo com o GitFlow: use nomes de cards coerentes com os nomes das branches, como feature/geracao-codigo-ia, feature/testes-automatizados, feature/pipeline-ci-cd e docs/prompts-arquitetura. Isso facilita a rastreabilidade entre planejamento, execução, versionamento e entrega.

Importante:

O quadro deve refletir o processo real de desenvolvimento do grupo. Cards criados apenas ao final do projeto, sem descrição adequada ou sem movimentação coerente, não demonstram acompanhamento efetivo das atividades.

Tarefas sugeridas para o quadro do GitHub:

- Criar repositório privado no GitHub e adicionar colaboradores
- Definir domínio e escopo da aplicação
- Planejar arquitetura com suporte de IA e documentar decisões
- Criar estrutura inicial do projeto e README.md
- Gerar código das funcionalidades principais com IA (ciclo 1)
- Avaliar saída da IA e aplicar refinamento (ciclo 2)
- Implementar terceiro ciclo com padrão de prompting diferente (ciclo 3)
- Refatorar código com suporte de IA (Clean Code / SOLID)
- Documentar refatoração com comparativo antes/depois e prompt
- Gerar suíte de testes com suporte de IA
- Validar e ajustar testes gerados
- Gerar documentação automática com IA (docstrings, Swagger ou README)
- Configurar pipeline de CI/CD com GitHub Actions via IA
- Testar e validar pipeline
- Salvar todos os prompts utilizados em docs/prompts/
- Documentar caso de análise crítica de saída incorreta da IA
- Atualizar README.md com diagrama, instruções e evidências
- Gravar vídeo de demonstração e publicar no YouTube como não listado
- Revisar checklist final e submeter links no AVA

5.4. USO DO REPOSITÓRIO NO GITHUB

Utilize um repositório GitHub privado para controle de versionamento do projeto. Siga as boas práticas abaixo:

- Faça commits com mensagens sucintas e diretas, com poucos arquivos alterados por commit. Prefira "implementa X" ou "corrige Y" em vez de "X foi implementado" ou "Y foi corrigido".
- Crie uma branch develop para concentrar os merges durante o desenvolvimento.
- Utilize feature branches a partir da develop para cada tarefa ou funcionalidade.
- Não exclua branches após o pull request concluído.
- O código final do projeto deve estar todo mergeado na branch main.

Branches mínimas obrigatórias:

- main
- develop
- feature/especificacao-arquitetura
- feature/geracao-codigo-ia

- feature/refatoracao-ia
- feature/testes-automatizados
- feature/pipeline-ci-cd
- docs/prompts-readme

Mínimo de 8 commits, com mensagens claras e representando avanços incrementais.

Exemplos de mensagens de commit:

- feat: cria estrutura inicial do projeto com arquitetura definida via IA
- feat: implementa funcionalidade principal gerada com Chain of Thought
- feat: adiciona segunda funcionalidade com refinamento de prompt documentado
- feat: refatora módulo principal seguindo SOLID com suporte de IA
- feat: gera suíte de testes unitários com apoio de IA
- feat: adiciona testes E2E cobrindo cenários principais
- feat: configura pipeline CI/CD com GitHub Actions via IA
- feat: adiciona documentação Swagger gerada automaticamente com IA
- docs: salva prompts organizados por etapa em docs/prompts/
- docs: documenta caso de saída incorreta da IA e refinamento aplicado
- docs: atualiza README com diagrama de arquitetura e instruções completas
- fix: corrige erro identificado na saída gerada pela IA.

5.5. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento da aplicação, o grupo deverá gravar um vídeo de até 10 minutos demonstrando o projeto e os artefatos produzidos ao longo do desenvolvimento.

O vídeo deverá ser publicado no YouTube como vídeo não listado e o link deverá ser inserido no README.md do repositório e submetido no AVA junto com os demais links do projeto. O vídeo é obrigatório e compõe a nota.

O vídeo deverá cobrir os seguintes pontos, mostrando os artefatos reais produzidos:

- demonstração do sistema em funcionamento, com ao menos dois cenários de uso;
- exibição do repositório GitHub: estrutura de pastas, branches criadas e histórico de commits;

- exibição do quadro do GitHub com os cards do projeto e seu andamento;
- exibição da pasta docs/prompts/ com os prompts organizados por etapa;
- demonstração de ao menos um ciclo completo de geração e refinamento de código com IA, mostrando o prompt utilizado e o resultado obtido;
- exibição da refatoração documentada: código antes, prompt utilizado e código após;
- demonstração dos testes gerados com IA sendo executados;
- exibição do pipeline de CI/CD configurado e funcionando no GitHub Actions;
- demonstração da documentação técnica gerada com IA (docstrings, Swagger ou README);
- apresentação do caso documentado de saída incorreta da IA, com análise crítica e correção aplicada.

6. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. A nota possui variação de 0 (zero) a 10 (dez) e peso de 60% sobre a avaliação do módulo (Avaliação M1.2).

Serão desconsiderados e atribuídos nota 0 (zero) a projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. Você está livre para consultar materiais e ferramentas de apoio, mas não é permitida a cópia.

Apresentação do Projeto			
Nº	Critério de Avaliação	0	1,00
1	O grupo entregou o vídeo de demonstração no YouTube (não listado), com até 10 minutos, cobrindo o sistema em funcionamento e os artefatos produzidos conforme descrito no item 4.5?	O grupo não entregou o vídeo, o vídeo está inacessível ou não demonstra os artefatos e o sistema solicitados..	O grupo entregou o vídeo acessível no YouTube, com até 10 minutos, demonstrando o sistema funcionando e cobrindo os artefatos descritos no item 4.5.

Uso adequado do quadro do GitHub				
Nº	Critério de Avaliação	0	0,25	0,50

2	O grupo organizou as tarefas do projeto em cards no quadro do GitHub?	O grupo não criou cards relacionados ao projeto ou o quadro não está acessível.	O grupo criou poucos cards ou os cards não refletem adequadamente o escopo do projeto.	O grupo criou cards coerentes com o escopo do projeto, cobrindo as principais etapas de desenvolvimento.
3	Os cards possuem descrição suficiente para entender a tarefa realizada?	Os cards não possuem descrição ou não permitem entender o que deveria ser feito.	Os cards possuem descrição superficial, com pouca clareza sobre objetivo, escopo ou resultado esperado.	Os cards possuem descrição clara da tarefa, objetivo, escopo e resultado esperado.
4	O grupo manteve o andamento dos cards atualizado durante o desenvolvimento?	O grupo não movimentou os cards ou movimentou apenas ao final do projeto.	O grupo movimentou os cards de forma irregular, sem refletir adequadamente o andamento do trabalho.	O grupo movimentou os cards de forma coerente com o fluxo Kanban, refletindo o andamento real das tarefas.

Uso adequado do GitHub e Readme.md

Nº	Critério de Avaliação	0	0,25	0,75
5	O repositório está organizado, privado e com os colaboradores obrigatórios adicionados?	O repositório não existe, está público sem justificativa ou não possui os colaboradores adicionados.	O repositório existe e está privado, mas apresenta problemas de organização ou os colaboradores não foram adicionados corretamente.	O repositório está privado, organizado, com os colaboradores obrigatórios adicionados e com estrutura compreensível.
6	O grupo utilizou branches e commits seguindo o fluxo de versionamento definido?	O grupo não utilizou branches ou realizou todas as alterações diretamente na main sem justificativa.	O grupo utilizou branches, mas de forma inconsistente, com nomes pouco descritivos ou sem relação clara com as tarefas.	O grupo utilizou branches adequadas para cada etapa, com nomes descritivos, respeitando o fluxo develop → feature → PR → main.
7	Os commits possuem mensagens claras e representam avanços incrementais?	Os commits são inexistentes, genéricos ou não permitem entender o que foi alterado.	Os commits existem, mas possuem mensagens pouco claras ou agrupam alterações grandes e desorganizadas.	Os commits possuem mensagens claras, objetivas e representam avanços incrementais no desenvolvimento.
8	O README.md está completo com todos os itens obrigatórios descritos no item 4.2?	O README.md está ausente, incompleto ou não permite entender o projeto e como executá-lo.	O README.md existe mas está parcialmente preenchido, com instruções vagas, itens ausentes ou sem evidências de uso de IA.	O README.md está completo, com nome, descrição, arquitetura, instruções de execução, ferramentas de IA utilizadas, padrões de prompting, cenários de uso, caso de análise crítica e links obrigatórios.

Desenvolvimento adequado da Aplicação

Nº	Critério de Avaliação	0	0,25	0,50
----	-----------------------	---	------	------

7	A aplicação é funcional, executável e demonstrável com ao menos dois cenários de uso?	A aplicação não executa, não produz resultado útil ou depende de dados completamente fixos no código.	A aplicação executa parcialmente, com falhas relevantes de funcionamento, integração ou demonstração.	A aplicação executa de forma coerente, recebe entradas e produz saídas funcionais em pelo menos dois cenários demonstráveis e documentados.
8	A arquitetura foi planejada e documentada com suporte de IA, com decisões justificadas?	Não há evidência de uso de IA no planejamento arquitetural ou as decisões não foram documentadas.	A arquitetura foi parcialmente assistida por IA, mas com decisões pouco claras, diagrama ausente ou justificativas genéricas.	A arquitetura foi planejada com suporte de IA, está documentada no README com diagrama ou descrição clara, e as principais decisões técnicas estão justificadas com referência aos prompts utilizados.
9	O código foi gerado e refinado com suporte de IA, com ao menos 3 ciclos completos documentados?	Não há evidências de uso de IA na geração de código ou os prompts não foram salvos.	Há evidências de uso de IA na geração de código, mas os ciclos de refinamento são incompletos, os prompts são genéricos ou a avaliação crítica está ausente.	Realizou pelo menos 3 ciclos completos de geração e refinamento documentados, com prompts salvos, saídas avaliadas criticamente e ajustes registrados.
10	Foram aplicados ao menos 2 padrões distintos de prompting com engenharia de contexto adequada?	Não foram aplicados padrões de prompting reconhecíveis ou os prompts são simples demais, sem estrutura de contexto.	Foi aplicado apenas 1 padrão ou os prompts possuem contexto parcial, com instruções vagas ou sem exemplos e restrições relevantes.	Foram aplicados pelo menos 2 padrões distintos — entre Chain of Thought, Few-shot, Role-based, ReAct ou Tree of Thought — com contexto bem estruturado, objetivos claros, exemplos e restrições explícitas.
11	O código passou por ao menos uma refatoração documentada com suporte de IA?	Não há evidência de refatoração ou a refatoração não foi documentada.	Há refatoração, mas sem documentação clara do prompt usado, do estado anterior, do resultado obtido ou da motivação técnica.	A refatoração está documentada com: prompt utilizado, estado anterior do código, resultado gerado, critério técnico aplicado (Clean Code, SOLID, otimização ou conversão arquitetural) e avaliação do resultado.
12	A aplicação possui suíte de testes gerada com suporte de IA, cobrindo cenários relevantes?	Não há testes ou não há qualquer evidência ou suporte de IA na sua criação.	Há testes gerados com IA, mas com cobertura superficial, casos triviais ou sem relação clara com as funcionalidades reais da aplicação.	A suíte de testes foi gerada com apoio de IA, cobrindo cenários relevantes — unitários, E2E ou de API — com prompts salvos e evidência de que os testes foram validados pelo grupo.
13	A aplicação possui documentação técnica	Não há documentação gerada com IA ou a documentação é	Há documentação parcial gerada com IA, mas sem cobertura adequada, com	A documentação cobriu adequadamente o projeto — docstrings, README completo,

	automática gerada com suporte de IA?	insuficiente para entender e executar o projeto.	prompts genéricos ou sem integração ao código e ao README.	Swagger/OpenAPI ou diagramas — com evidências dos prompts usados e resultado integrado ao repositório.
14	A aplicação possui pipeline de CI/CD configurado com suporte de IA, executando lint e testes automaticamente?	Não há pipeline configurado ou ele não executa de forma demonstrável.	Há pipeline parcialmente configurado, mas com falhas de execução, cobertura mínima ou sem evidência de uso de IA na sua criação.	O pipeline de CI/CD está configurado e funcional, executa ao menos lint e testes automaticamente a cada push, foi criado com suporte de IA e está documentado com os prompts utilizados.
15	O grupo demonstrou análise crítica sobre ao menos uma saída da IA que precisou ser corrigida ou refinada?	Não há evidência de avaliação crítica; toda saída da IA foi aceita sem questionamento documentado.	Há menção a correções, mas sem documentação clara do problema identificado, da correção aplicada ou da justificativa técnica.	O grupo documentou ao menos um caso em que a saída da IA foi identificada como incorreta ou insuficiente, com descrição do problema, da correção realizada e da lição aprendida.

7. CHECKLIST FINAL DE ENTREGA

Antes de submeter no AVA, confira:

Repositório e organização:

- ☐ Criei o repositório privado no GitHub com os colaboradores obrigatórios adicionados
- ☐ Criei a estrutura de pastas do projeto com README.md, docs/prompts/ e .env.example
- ☐ Criei o quadro Kanban no GitHub com as 6 colunas obrigatórias
- ☐ Criei cards no quadro do GitHub com descrição clara para cada etapa do desenvolvimento
- ☐ Mantive os cards atualizados ao longo do projeto
- ☐ Usei branches descritivas a partir da develop para cada funcionalidade
- ☐ Fiz ao menos 8 commits com mensagens claras e incrementais
- ☐ Mergei tudo na branch main ao final

Desenvolvimento com IA:

- ☐ Defini e documentei o domínio e escopo da aplicação
- ☐ Planejei a arquitetura com suporte de IA e documentei as decisões no README

- ☐ Documentei ao menos 3 ciclos completos de geração e refinamento de código
- ☐ Apliquei ao menos 2 padrões distintos de prompting com contexto estruturado
- ☐ Salvei todos os prompts utilizados, organizados por etapa, em docs/prompts/
- ☐ Documentei ao menos 1 refatoração com suporte de IA (prompt + antes + depois + critério)
- ☐ Gerei suíte de testes com apoio de IA e validei os testes gerados
- ☐ Gerei documentação automática com IA (docstrings, Swagger, README ou diagramas)
- ☐ Configurei pipeline de CI/CD com GitHub Actions via IA, executando lint e testes
- ☐ Documentei ao menos 1 caso de saída incorreta da IA com análise crítica e correção
- ☐ Demonstrei ao menos 2 cenários de uso da aplicação

README.md:

- ☐ Nome do projeto e descrição do problema resolvido
- ☐ Ferramentas de IA utilizadas e em quais etapas
- ☐ Padrões de prompting aplicados com exemplos
- ☐ Diagrama ou descrição da arquitetura
- ☐ Instruções completas de instalação e execução
- ☐ Cenários de uso com exemplos de entrada e saída
- ☐ Caso documentado de saída incorreta da IA
- ☐ Melhorias futuras
- ☐ Link do vídeo no YouTube (não listado)

Vídeo de demonstração:

- ☐ Gravei o vídeo de até 10 minutos
- ☐ O vídeo demonstra o sistema em funcionamento com ao menos dois cenários de uso
- ☐ O vídeo exhibe o repositório GitHub (estrutura, branches e histórico de commits)
- ☐ O vídeo exhibe o quadro GitHub com os cards do projeto
- ☐ O vídeo exhibe a pasta docs/prompts/ com os prompts organizados

- ☐ O vídeo mostra ao menos um ciclo completo de geração e refinamento de código com IA
- ☐ O vídeo exibe a refatoração documentada (antes, prompt e depois)
- ☐ O vídeo mostra os testes sendo executados
- ☐ O vídeo exibe o pipeline de CI/CD funcionando no GitHub Actions
- ☐ O vídeo mostra a documentação técnica gerada com IA
- ☐ O vídeo apresenta o caso de análise crítica de saída incorreta da IA
- ☐ Publiquei o vídeo no YouTube como não listado
- ☐ Inseri o link do vídeo no README.md

Submissão:

- ☐ Submeti o link do repositório GitHub no AVA
- ☐ Submeti o link do quadro do GitHub no AVA
- ☐ Realizei a submissão antes do prazo: 01/06/2026 às 15h